EEE518 - Digital Image Processing

Spring 2026

Homework 3: Filtering in Frequency Domain and Image Restoration

Student Name:	Veysel Yusuf Yilmaz
Student Number:	Y240246012
Course:	EEE518 - Digital Image Processing
Date:	12/04/2026

Question 1: Noise Removal

Problem: Remove the noise from the input images HW3_1_1, HW3_1_2, HW3_1_3, and HW3_1_4. Submit your code and the output images.

1.1 - HW3_1_1: Pepper Noise Removal (Contra-Harmonic Mean Filter, Q=1.5)

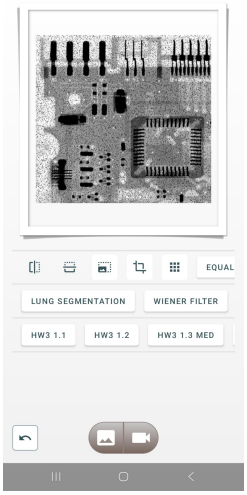
Method: The contra-harmonic mean filter is: $\mathbf{f(x,y) = [\Sigma\ g(s,t)^{Q+1}] / [\Sigma\ g(s,t)^{Q}]}$. With Q=1.5 and a 3×3 window, the filter emphasizes brighter pixels, eliminating pepper (dark) noise. Epsilon (10^{-10}) guards against division by zero when Q<0.

Kotlin Code:

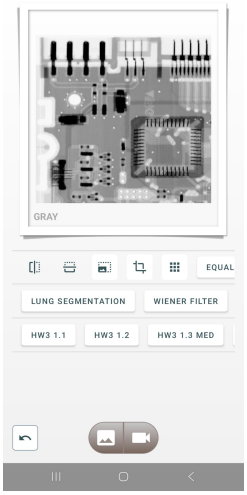
```
fun contraHarmonicFilter(bitmap: Bitmap, resources: Resources,
                         Q: Double, filterSize: Int): BitmapDrawable {
    val width = bitmap.width; val height = bitmap.height
    val pixels = IntArray(width * height)
    bitmap.getPixels(pixels, 0, width, 0, 0, width, height)
    val newPixels = IntArray(width * height); val half = filterSize / 2
    val epsilon = 1e-10
    for (y in 0 until height) {
        for (x in 0 until width) {
            var numerator = 0.0; var denominator = 0.0
            for (i in -half..half) {
                for (j in -half..half) {
                    val ny = y + i; val nx = x + j
                    if (ny < 0 || ny >= height || nx < 0 || nx >= width) continue
                    val g = (pixels[ny * width + nx] shr 16 and 0xff).toDouble()
                    if (Q < 0 && g == 0.0) {
                        numerator += (g + epsilon).pow(Q + 1)
                        denominator += (g + epsilon).pow(Q)
                    } else {
                        numerator += g.pow(Q + 1); denominator += g.pow(Q)
                    }
                }
            }
            val result = if (denominator != 0.0) numerator / denominator else 0.0
            val gray = result.toInt().coerceIn(0, 255)
            newPixels[y * width + x] =
                0xff000000.toInt() or (gray shl 16) or (gray shl 8) or gray
        }
    }
    val bmp = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)
    bmp.setPixels(newPixels, 0, width, 0, 0, width, height)
    return bmp.toDrawable(resources)
}
// HW3_1_1: Q=1.5, 3x3 -> removes pepper (dark) noise
fun hw3_1_1(bitmap: Bitmap, resources: Resources) =
```

```
    contraHarmonicFilter(bitmap, resources, Q = 1.5, filterSize = 3)
```

Output Images:



(a) Input (Pepper Noise)



(b) Result (Q=1.5, 3×3)

Discussion: The contra-harmonic mean filter with Q=1.5 successfully removes pepper noise while preserving image details. The positive Q value favors brighter pixels, replacing dark noise with values consistent with the surrounding region.

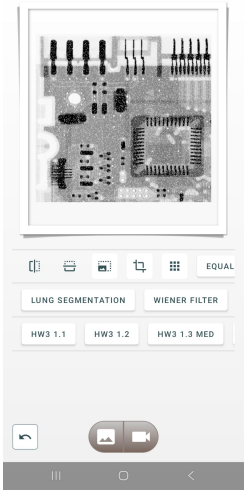
1.2 - HW3_1_2: Salt Noise Removal (Contra-Harmonic Mean Filter, Q=-1.5)

Method: Same contra-harmonic filter with Q=-1.5 emphasizes darker pixels, eliminating salt (bright) noise. Epsilon guard prevents division by zero for zero-valued pixels.

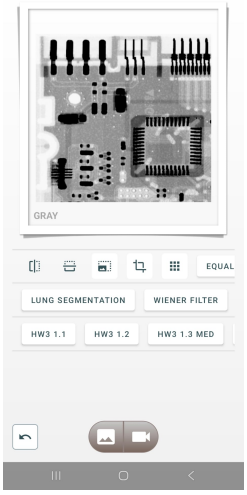
Kotlin Code:

```
// HW3_1_2: Q=-1.5, 3x3 -> removes salt (bright) noise
fun hw3_1_2(bitmap: Bitmap, resources: Resources) =
    contraHarmonicFilter(bitmap, resources, Q = -1.5, filterSize = 3)
```

Output Images:



(a) Input (Salt Noise)



(b) Result (Q=-1.5, 3×3)

Discussion: The negative Q (-1.5) effectively removes salt noise by favoring darker pixel values. This demonstrates the dual nature of the contra-harmonic filter: positive Q removes dark noise, negative Q removes bright noise.

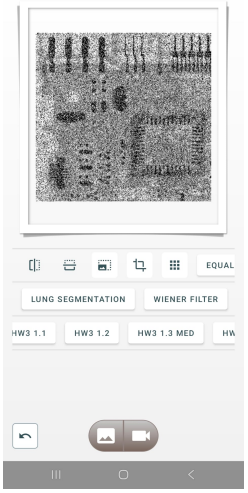
1.3 - HW3_1_3: Salt & Pepper Noise (Median vs. Adaptive Median)

Method: Two approaches compared: **(a) Median Filter (7×7)** replaces each pixel with the neighborhood median. **(b) Adaptive Median (S_{max}=7)** starts at 3×3, expands until a non-impulse median is found. Level A checks if median is impulse; Level B checks if center pixel is impulse. Preserves more detail than fixed-size median.

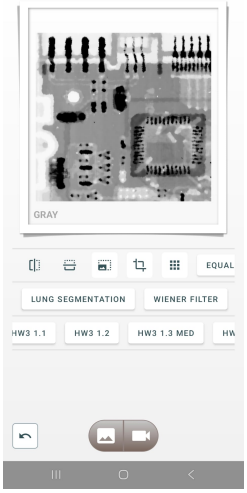
Kotlin Code:

```
fun medianFilterNxN(bitmap: Bitmap, resources: Resources, filterSize: Int)
    : BitmapDrawable {
    val width = bitmap.width; val height = bitmap.height
    val pixels = IntArray(width * height)
    bitmap.getPixels(pixels, 0, width, 0, 0, width, height)
    val newPixels = IntArray(width * height); val half = filterSize / 2
    for (y in 0 until height) {
        for (x in 0 until width) {
            val values = mutableListOf<Int>()
            for (i in -half..half) for (j in -half..half) {
                val ny = y + i; val nx = x + j
                if (ny in 0..<height && nx in 0..<width)
                    values.add(pixels[ny * width + nx] shr 16 and 0xff)
            }
            values.sort(); val gray = values[values.size / 2]
            newPixels[y*width+x] = 0xff000000.toInt() or (gray shl 16) or (gray shl 8) or gray
        }
    }
    // ... create output bitmap
}
fun hw3_1_3Median(bitmap: Bitmap, resources: Resources) =
    medianFilterNxN(bitmap, resources, filterSize = 7)
fun adaptiveMedianFilter(bitmap: Bitmap, resources: Resources, sMax: Int)
    : BitmapDrawable {
    // ... for each pixel:
    var windowSize = 3; var outputGray = zxy
    while (windowSize <= sMax) {
        // collect neighborhood, sort -> zmin, zmax, zmed
        if (zmed - zmin > 0 && zmed - zmax < 0) {       // Level A: median OK
            outputGray = if (zxy-zmin > 0 && zxy-zmax < 0) zxy else zmed  // Level B
            break
        } else { windowSize += 2 }  // expand window
        if (windowSize > sMax) { outputGray = zmed; break }
    }
}
fun hw3_1_3Adaptive(bitmap: Bitmap, resources: Resources) =
    adaptiveMedianFilter(bitmap, resources, sMax = 7)
```

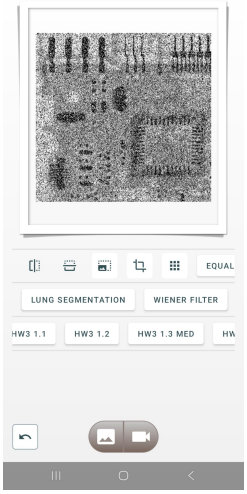
Output Images:



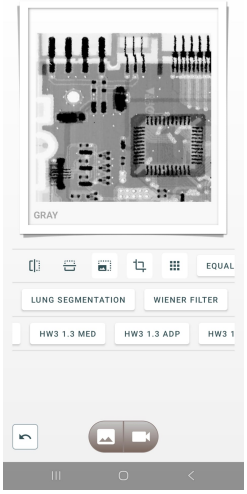
(a) Input (Salt & Pepper Noise)



(b) Median Filter (7×7)



(a) Input (Salt & Pepper Noise)



(c) Adaptive Median (S_{max}=7)

Discussion: Both filters remove salt & pepper noise effectively, but the adaptive median preserves significantly more detail. The fixed 7×7 median produces a smoother but blurrier result. The adaptive filter starts small (3×3) and only enlarges when the median is corrupted, preserving edges and fine details in clean regions.

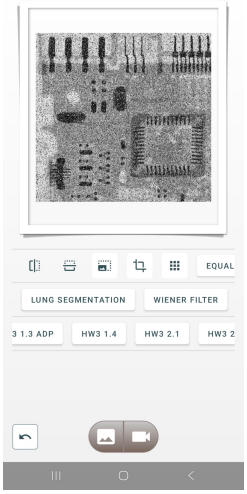
1.4 - HW3_1_4: Uniform Noise Removal (Repeated 3×3 Median)

Method: A 3×3 median filter applied three times iteratively. Each pass removes more noise while the small kernel preserves edges better than a single large kernel.

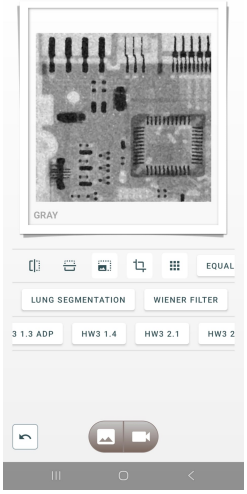
Kotlin Code:

```
fun hw3_1_4(bitmap: Bitmap, resources: Resources): BitmapDrawable {
    val first = medianFilterNxN(bitmap, resources, filterSize = 3)
    val second = medianFilterNxN(first.bitmap, resources, filterSize = 3)
    return medianFilterNxN(second.bitmap, resources, filterSize = 3)
}
```

Output Images:



(a) Input (Uniform Noise)



(b) Result (3×3 Median ×3)

Discussion: The iterative 3×3 median filtering effectively reduces uniform noise while preserving structural edges. Each iteration progressively smooths noise, and the small kernel prevents over-blurring that a 7×7 or larger filter would cause.

Question 2: Image Restoration

Problem: Restore the original images from the inputs HW3_2_1, HW3_2_2 and HW3_2_3.

Background: Images are degraded by motion blur: $\mathbf{H(u,v) = (T/\pi\alpha)\cdot\sin(\pi\alpha)\cdot e^{-j\pi\alpha}}$, $\alpha = ua + vb$. Parameters: a=b=0.1, T=1. Complex form: $H_{real} = T\cdot\sin(\alpha)\cdot\cos(\alpha)/\alpha$, $H_{imag} = -T\cdot\sin^2(\alpha)/\alpha$, with H=T when $\alpha\to 0$. For S&P noise images (2_1, 2_2), 5×5 median denoising is applied first.

Kotlin Code (Core Functions):

```
private const val MOTION_A = 0.1
private const val MOTION_B = 0.1
private const val MOTION_T = 1.0
private const val WIENER_K_NOISY = 0.01   // for S&P noise images
private const val WIENER_K_CLEAN = 0.005  // for motion-only image
private const val INVERSE_RADIUS_RATIO = 0.3
private const val INVERSE_TRANSFER_THRESHOLD = 1e-3
// Build H(u,v) as 2-channel complex Mat (centered frequencies)
private fun buildMotionDegradationH(rows: Int, cols: Int,
    a: Double, b: Double, T: Double): Mat {
    for (u in 0 until rows) for (v in 0 until cols) {
        val alpha = PI * ((u - rows/2.0)*a + (v - cols/2.0)*b)
        if (abs(alpha) < 1e-10) { hReal=T; hImag=0 }
        else { hReal = T*sin(alpha)*cos(alpha)/alpha
               hImag = -T*sin(alpha)^2/alpha }
    }
    return merge(realMat, imagMat)
}
// Wiener: F = [H*/(|H|^2+K)] * G
private fun applyWienerFilter(complexG: Mat, complexH: Mat, K: Double): Mat
// Inverse: G/H with radius limit & threshold guard
private fun applyInverseFilter(complexG: Mat, complexH: Mat, ...): Mat
// DFT pipeline (no padding), IDFT with DFT_SCALE, saturating cast
private fun bitmapToDftComplex(bitmap: Bitmap): Pair<Mat, Pair<Int,Int>>
private fun complexToResultBitmap(complex: Mat, ...): BitmapDrawable
```

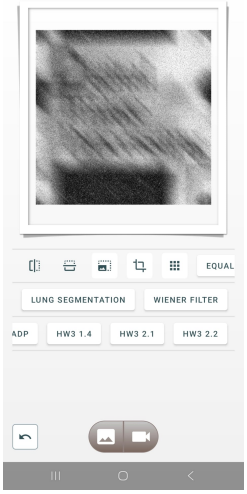
2.1 - HW3_2_1: Motion Blur + Heavy S&P Noise

Pipeline: Step 1: 5×5 median denoise. Step 2: Wiener filter (K=0.01).

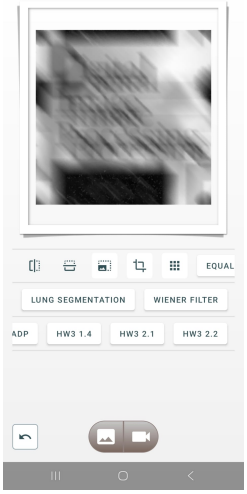
Kotlin Code:

```
fun hw3_2_1(bitmap: Bitmap, resources: Resources): List<BitmapDrawable> {
    val denoised = denoiseMedian(bitmap, DENOISE_FILTER_SIZE, resources)
    val (complexG, origSize) = bitmapToDftComplex(denoised.bitmap)
    val H = buildMotionDegradationH(complexG.rows(), complexG.cols(),
                                    MOTION_A, MOTION_B, MOTION_T)
    val restored = applyWienerFilter(complexG, H, WIENER_K_NOISY)
    shiftDFT(restored)
    val restoredBmp = complexToResultBitmap(restored, origSize.first,
                                            origSize.second, resources)
    return listOf(denoised, restoredBmp)
}
```

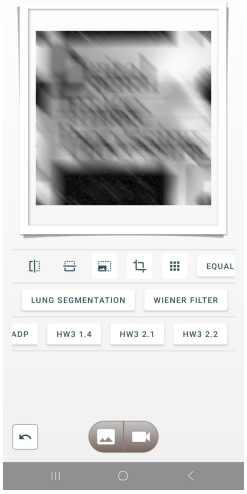
Output Images:



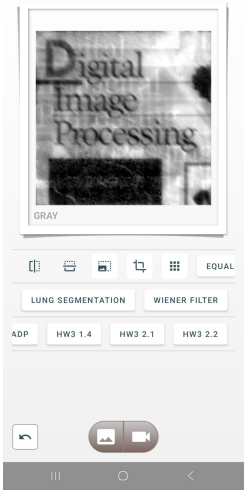
(a) Input (Motion + Heavy S&P)



(b) Step 1: Median Denoised



(b) Step 1: Median Denoised



(c) Step 2: Wiener Restored

Discussion: The two-stage pipeline first removes S&P noise with a 5×5 median, then applies Wiener filtering to deconvolve motion blur. Denoising is critical because Wiener would amplify impulse noise in frequency domain. K=0.01 balances noise suppression vs. detail recovery.

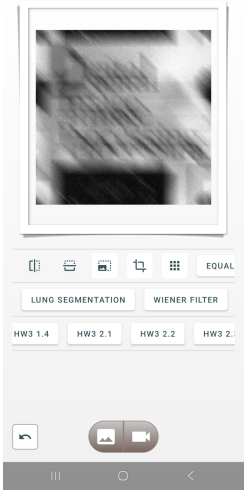
2.2 - HW3_2_2: Motion Blur + Mild S&P Noise

Pipeline: Same as 2.1: 5×5 median denoise, then Wiener (K=0.01).

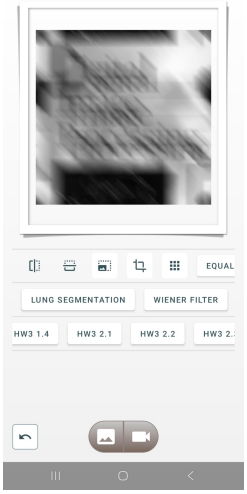
Kotlin Code:

```
fun hw3_2_2(bitmap: Bitmap, resources: Resources): List<BitmapDrawable> {
    val denoised = denoiseMedian(bitmap, DENOISE_FILTER_SIZE, resources)
    val (complexG, origSize) = bitmapToDftComplex(denoised.bitmap)
    val H = buildMotionDegradationH(complexG.rows(), complexG.cols(),
                                    MOTION_A, MOTION_B, MOTION_T)
    val restored = applyWienerFilter(complexG, H, WIENER_K_NOISY)
    shiftDFT(restored)
    val restoredBmp = complexToResultBitmap(restored, origSize.first,
                                            origSize.second, resources)
    return listOf(denoised, restoredBmp)
}
```

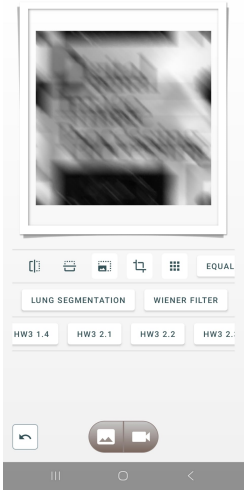
Output Images:



(a) Input (Motion + Mild S&P)



(b) Step 1: Median Denoised



(b) Step 1: Median Denoised

(c) Step 2: Wiener Restored

Discussion: With milder S&P noise, median denoising preserves more detail, leading to a cleaner Wiener input. The restoration shows better sharpness compared to HW3_2_1.

2.3 - HW3_2_3: Motion Blur Only (Inverse & Wiener Filters)

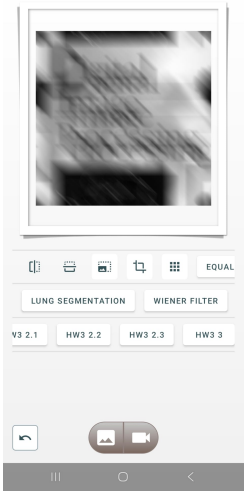
Pipeline: Step 1: Radius-limited inverse filter. Step 2: Wiener (K=0.005). No denoising needed.

Kotlin Code:

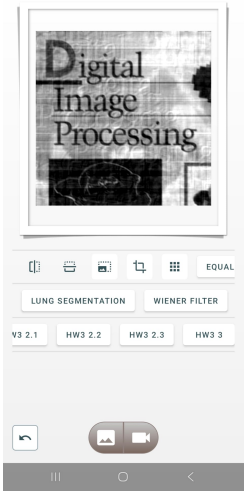
```
fun hw3_2_3(bitmap: Bitmap, resources: Resources): List<BitmapDrawable> {
    val (complexG, origSize) = bitmapToDftComplex(bitmap)
    val H = buildMotionDegradationH(complexG.rows(), complexG.cols(),
                                    MOTION_A, MOTION_B, MOTION_T)
    // Step 1: Inverse filter (radius-limited, threshold-guarded)
    val inversed = applyInverseFilter(complexG, H, complexG.rows(), complexG.cols())
    shiftDFT(inversed)
    val inverseBmp = complexToResultBitmap(inversed, origSize.first,
                                           origSize.second, resources)
    // Step 2: Wiener with lower K (motion-only, no noise)
    val (complexG2, _) = bitmapToDftComplex(bitmap)
    val H2 = buildMotionDegradationH(complexG2.rows(), complexG2.cols(),
                                     MOTION_A, MOTION_B, MOTION_T)
    val wienered = applyWienerFilter(complexG2, H2, WIENER_K_CLEAN)
    shiftDFT(wienered)
    val wienerBmp = complexToResultBitmap(wienered, origSize.first,
```

```
                                        origSize.second, resources)
    return listOf(inverseBmp, wienerBmp)
}
```

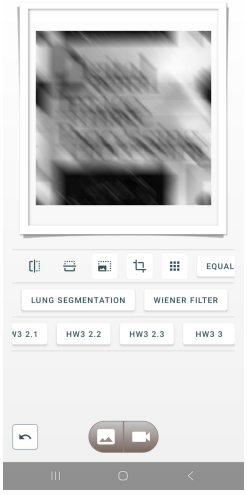
Output Images:



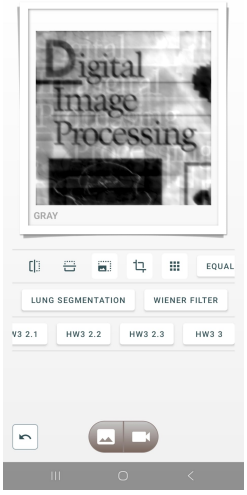
(a) Input (Motion Blur Only)



(b) Step 1: Inverse Filter



(a) Input (Motion Blur Only)



(c) Step 2: Wiener Filter

Discussion: Without additive noise, both filters apply directly. The inverse filter uses radius limitation (30% of image half-size) and threshold (10^{-3}) to prevent amplification where $|H|\approx 0$. Wiener with K=0.005 provides less regularization, recovering more detail. Wiener generally produces a more stable result; the inverse filter can exhibit ringing at the radius boundary.

Question 3: Fingerprint Enhancement

Problem: Implement the procedure from lecture 5&6, page 103. Input: HW3_3.

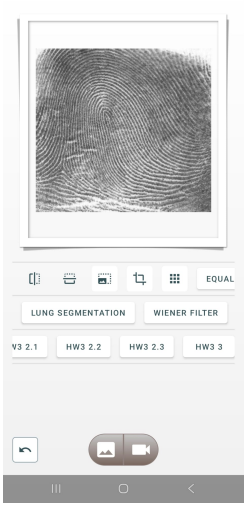
Method: Butterworth High-Pass Filter (BHPF) in frequency domain enhances fingerprint ridges, followed by Otsu thresholding. Transfer function: $\mathbf{H(u,v) = 1 / [1 + (D_0/D(u,v))^{2n}]}$, D_0=50, n=4. Pipeline: pad → DFT → shift → multiply by H → shift back → IDFT → normalize → Otsu threshold.

Kotlin Code:

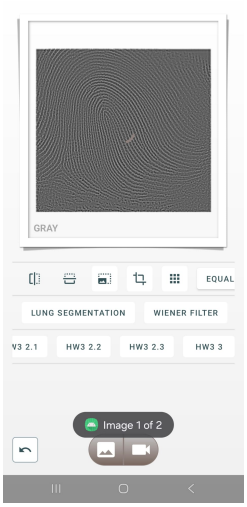
```
private const val BHPF_ORDER = 4
```

```
private const val BHPF_CUTOFF = 50.0
private fun butterworthHighPassFilter(bitmap: Bitmap, resources: Resources,
    order: Int, cutoff: Double): BitmapDrawable {
    val image = Mat()
    Utils.bitmapToMat(bitmap, image)
    Imgproc.cvtColor(image, image, Imgproc.COLOR_BGR2GRAY)
    // Pad to optimal DFT size
    val m = Core.getOptimalDFTSize(image.rows())
    val n = Core.getOptimalDFTSize(image.cols())
    Core.copyMakeBorder(image, padded, 0, m-image.rows(),
                        0, n-image.cols(), Core.BORDER_CONSTANT, Scalar.all(0.0))
    // Create complex image, compute DFT, shift to center
    Core.dft(complexImage, complexImage); shiftDFT(complexImage)
    // Build BHPF
    for (u in 0 until rows) for (v in 0 until cols) {
        val d = sqrt((u-centerY).pow(2) + (v-centerX).pow(2))
        val h = if (d==0.0) 0.0 else 1.0/(1.0 + (cutoff/d).pow(2.0*order))
        filterMat.put(u, v, h)
    }
    Core.mulSpectrums(complexImage, filterMat, filtered, 0)
    shiftDFT(filtered); Core.idft(filtered, filtered)
    Core.normalize(cropped, restoredImage, 0.0, 255.0, Core.NORM_MINMAX)
    restoredImage.convertTo(restoredImage, CvType.CV_8UC1)
    // ... create output bitmap
}
private fun thresholdImage(bitmap: Bitmap, resources: Resources): BitmapDrawable {
    Imgproc.threshold(mat, binary, 0.0, 255.0,
                      Imgproc.THRESH_BINARY + Imgproc.THRESH_OTSU)
    // ... create output bitmap
}
fun hw3_3Process(bitmap: Bitmap, resources: Resources): List<BitmapDrawable> {
    val highPassed = butterworthHighPassFilter(bitmap, resources,
                                               BHPF_ORDER, BHPF_CUTOFF)
    val thresholded = thresholdImage(highPassed.bitmap, resources)
    return listOf(highPassed, thresholded)
}
```

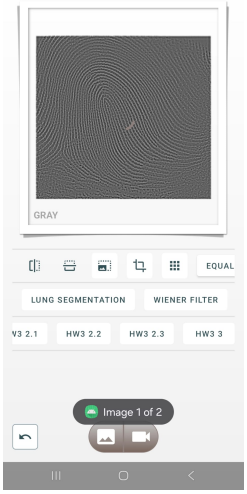
Output Images:



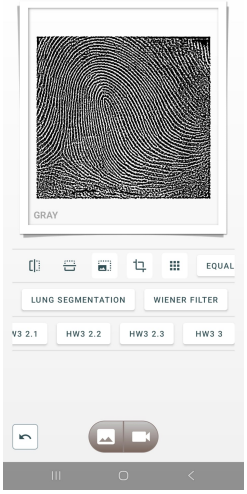
(a) Input (Fingerprint)



(b) Step 1: Butterworth HPF



(b) Step 1: Butterworth HPF



(c) Step 2: Otsu Threshold

Discussion: The 4th-order Butterworth HPF with D_0=50 enhances fingerprint ridges by attenuating low-frequency background variations while preserving high-frequency edges. The smooth transition avoids ringing artifacts of an ideal HPF. Otsu thresholding produces a clean binary image clearly separating ridges from background, suitable for further analysis or matching.